

Small workflow runners and one observing server

Design proposal for Gimbal issue 439 · October 9, 2026

Execution and data flow

Compile project workflows into a small executable. Run each invocation in its own process. Let the central server subscribe to that process over HTTP, with events streamed as they happen and controls sent as ordinary requests. Use Unix sockets locally. The same application protocol can later use a network connection. Tens of runs need neither a broker nor a scheduling service.

The server initiates the connection; the runner pushes events down the open response. This is a subscription, not repeated polling. It gives us one direction for connection establishment, straightforward request/reply controls, and ordinary Go net/http handlers. Runner-initiated uploads would also work, but would make every runner manage a destination and reconnect policy while still requiring a return path for controls. Choose subscriptions while the server can reach the runners.

Browser / CLI → central server → runner: controls and artifact reads
→ runner: open event subscription
← runner: ordered events, continuously

The runner owns the workflow stack, typed `Generate[T]` results, agent sessions, cancellation, cleanup, and recording. The server owns project discovery, launching, the run index, observation reduction, browser snapshots, and browser SSE. Neither browser connections nor IPC request contexts own the workflow lifetime. Cancelling a run is an explicit command.

Keep the web application out of the runner's import graph. A generated project entry point links its workflows, runtime, selected adapters, recorder, and small HTTP handler. It does not import web, skgo, embedded frontend assets, the built-in workflow catalog, or the source analyzer. Build separate entry points: the installed CLI/server and the project runner. A server-mode flag alone does not remove linked packages or their initialization; a separate build excludes them. A web plugin adds [Go toolchain and dependency matching constraints](#) without improving this boundary. Build-time analysis emits command descriptions and design data as sidecar files; typed schemas stay with executable workflow code.

The shared CLI constructs commands from that description, preserving flags, help, validation, and role selection; the executable validates inputs again before execution. The server renders project-specific design metadata through its common frontend. Keep `RegisterGraph/RegisteredGraph` runner-local. Replace the server's registration check and name-only lookup with saved design metadata keyed by project, workflow, and build; each run references that exact copy. A project's `.gimbal/workflows.json` lists entry packages. Proposed gimbal workflows refresh runs analysis and go build, publishes the runner and sidecars under `.gimbal/builds/<build>/`, then atomically updates a current-build pointer read by CLI and server. Edits take effect on explicit refresh; prebuilt bundles use the same boundary.

Removing web assets alone does not solve memory growth. Today `run.go` creates an observation Store even without a server; it retains completed turns and transcripts. **Move that projection to the central server.** The runner writes a new `events.jsonl`: lifecycle and agent payloads with placement, native references, and one monotonic journal position. Existing `run.jsonl` and session logs retain their per-log Seq meanings and human-readable records. This deliberately duplicates recording on disk to provide one ordered IPC replay source without retaining UI state. Derived observation snapshots and deltas move to the server.

One writer orders complete records. Disk-backed streaming uses bounded buffers; slow readers cannot block agent execution or accumulate unbounded queues. Runners retain active runtime and adapter state, not historical UI projections.

The launcher also supplies context-store and cache paths; the runner constructs `ContextAccess` locally. Immutable, content-addressed objects may be shared across local runs; their mutable execution files remain exclusive. Local artifact serving keeps `ReadObject`. Remote runners write to their own object store and expose referenced objects through artifact reads into the server's archive. Agent-visible materialization stays on the runner's machine.

Discovery, recovery, and controls

The central launcher reserves the run ID and directory and passes them with inputs and bindings. Revise Run/OpenRun to accept assigned identity for hosted execution; direct library calls still allocate their own identity and run in-process. Replace the unsupported-sharing caveat with explicit ownership of each run's mutable files. Runners stop writing project.jsonl; the server maintains the project index from their events and retains owner.lock. CLI launches go through this launcher. The server saves reservations and directory paths in its run index. The runner writes runner.json in its assigned directory with endpoint/build identity. Server startup reads these descriptors and verifies identity live before reconnecting.

GET events streams newline-delimited JSON journal records after a supplied position, then waits for new records without a replay/live gap. The server persists its prefix under its own instance directory, projects/<project>/runs/<run>/events.jsonl, before advancing its cursor. It reduces that copy into the existing browser snapshot-plus-SSE contract. Reconnect resumes after that cursor; repeated positions are ignored and gaps are exposed. Runner sequence numbers and browser projection cursors are separate. A fresh server can replay from the beginning; snapshots and compaction can wait until replay cost justifies them.

Launch runners in their own OS session/process group with separate log files. Server shutdown leaves them running and does not wait. They finish independently; an operator can reconnect the server or send SIGTERM, handled as whole-run cancellation. There is no automatic orphan timeout.

A broken connection marks the run **unreachable**. Heartbeats detect idle connection loss. Terminal events record the workflow result, while cleanup status determines whether controls must remain available. Keep the runner reachable while owned cleanup is pending. After local exit, import the journal's missing suffix; restart replays the server copy. Also import direct-run journals from project run directories as recorded history, without asserting live ownership. Lost observation connectivity does not cancel execution. Failure to write the runner's authoritative journal retains the public Run contract: recording errors enter the returned run error. Server projection failures are reported separately.

The IPC surface follows controls we already have:

Exchange	Meaning
Identity and status	Verify run/build/protocol identity; report execution and cleanup state.
Event subscription	Lifecycle, transcript events, usage, pending interviews, and control outcomes.
Session steer; turn interrupt	Target current session/turn IDs; preserve landed, dropped, and error distinctions.
Loop steer; interview answer	Queue planner input or answer the exact pending question ID.
Whole-run cancel	Request cancellation and retry retained cleanup; acceptance is not proof of cessation.
Artifact read	Fetch referenced values by run-relative identity.

The IPC handler accepts a command ID, calls the existing controller, then appends a control_result journal envelope with that ID and the returned outcome/error. This leaves public lifecycle payloads and adapter signatures unchanged. After a timeout, match the ID against the journal; absence means unknown. Do not automatically retry steers or answers. Cancellation acceptance still differs from cleanup completion. No durable command queue is needed. Reconcile uncertain launches by their reserved run ID before launching again.

Remote runners keep paths local and transfer events and artifacts through the protocol. Their persistent storage must survive container exit until the server imports it. Remote launch, archive transfer, and deletion policy are later work. Authenticated HTTPS or a tunnel can carry the same requests. Outbound-only networks may need a runner-initiated tunnel; event and control semantics stay the same.

The first implementation should prove discovery, launch, live observation, interview/steer/cancel, history replay, and two projects sharing a workflow name. Check dependencies and measure idle and transcript-heavy runner memory separately from agent subprocesses. Demonstrate that a slow or disconnected observer does not stall the workflow or grow its IPC queue. Keep remote deployment, mixed-version compatibility, and seamless upgrades outside that first gate; reject incompatible protocol versions clearly.